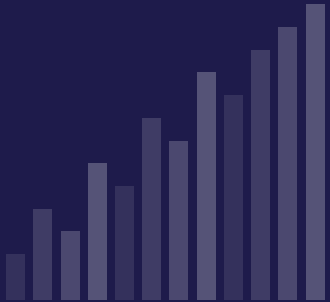
AULA 28 · BLOCO 4: GRAFOS E TÉCNICAS DE PROJETO

Grafos e busca em largura

Cormen et al., *Algoritmos*: seções 22.1 e 22.2



TEORIA

Roteiro da aula

Ao final, você será capaz de

- Representar grafos por listas de adjacência e por matriz de adjacência
- Executar a busca em largura e obter distâncias e a árvore de caminhos mínimos
- Provar que a BFS custa $O(V + E)$
- Medir o custo linear e o “mundo pequeno de grafos aleatórios

Teoria

Representações, BFS, distâncias, árvore de busca em largura

Prática em C

BFS em C com o grafo da Figura 22.3

Experimento

Operações $= V + 2E$; distância média em grafos aleatórios

Conexões

Redes sociais, web, roteamento e jogos



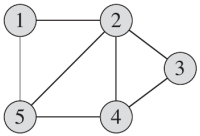
Teoria

- Cormen et al., seções 22.1 e 22.2
- Duas representações
- Busca em largura
- BFS passo a passo

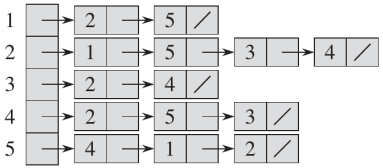
TEORIA

Duas representações

Listas: memória $\Theta(V + E)$; ótimas para grafos **esparsos** ($E \ll V^2$), como quase todos os reais. **Matriz:** memória $\Theta(V^2)$; responde "existe a aresta (u, v)?" em $O(1)$; boa para grafos **densos**.



(a)



(b)

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

(c)

Figura 22.1 de Cormen et al. (2009) — (a) grafo não direcionado; (b) listas de adjacência; (c) matriz de adjacência.

TEORIA

Busca em largura

- Explora em **camadas**: todos à distância k antes de qualquer um à distância $k+1$.
- Fila FIFO (aula 16).
- Cada vértice entra na fila uma vez; cada lista é percorrida uma vez: $O(V+E)$.
- **Teorema 22.5:** $v.d = \delta(s, v)$, a distância mínima em arestas.

BFS(G, s)

```
1  para cada u ∈ G.V − {s} faça
2      u.cor ← BRANCO;  u.d ← ∞;  u.π ← NIL
3  s.cor ← CINZA;  s.d ← 0;  Q ← {s}
4  enquanto Q ≠ ∅ faça
5      u ← Dequeue(Q)
6      para cada v ∈ G.Adj[u] faça
7          se v.cor = BRANCO então
8              v.cor ← CINZA;  v.d ← u.d + 1;  v.π ← u
9              Enqueue(Q, v)
10     u.cor ← PRETO
```

TEORIA

BFS passo a passo

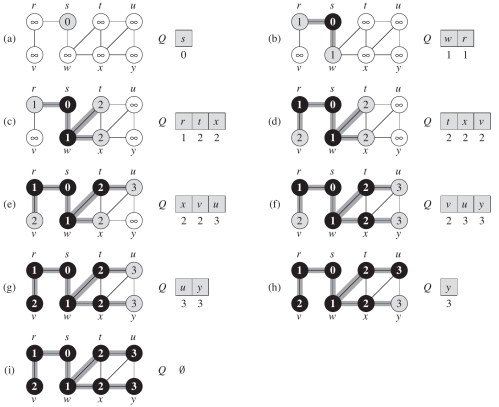


Figura 22.3 de Cormen et al. (2009) — BFS a partir de s: cinza = na fila, preto = terminado; Q mostra a fila e as

Prática em C

- Listas de adjacência com inserção na cabeça
- Fila em vetor: cada vértice entra uma vez
- Teste com o grafo da Figura 22.3 (vértices r a y)

</> PRÁTICA EM C

Busca em largura em C (1/4)

```c
#include <stdio.h>
#include <stdlib.h>

/* Entrada: V E, depois E arestas não direcionadas como pares de letras (a, b, ...),
   depois o índice da origem s (0 = primeira letra). Vértices: 'a' + i, ou as letras
   r s t u v w x y da Figura 22.3 quando V = 8. */
typedef struct Aresta { int v; struct Aresta *prox; } Aresta;

int V;
Aresta **adj;                                   // adj[u]: lista de vizinhos de u
```

PRÁTICA EM C

Busca em largura em C (2/4)

```c
void liga(int u, int v) {                    // insere na cabeça: O(1)
    Aresta *a = malloc(sizeof(Aresta));
    a->v = v;
    a->prox = adj[u];
    adj[u] = a;
}

int letra_para_vertice(char c, const char *nomes) {
    for (int i = 0; i < V; i++) if (nomes[i] == c) return i;
    return -1;
}

int main(void) {
    int E, s;
    if (scanf("%d %d", &V, &E) != 2) return 1;
    const char *nomes = V == 8 ? "rstuvwxy" : "abcdefghijklmnopqrstuvwxyz";
```

PRÁTICA EM C

Busca em largura em C (3/4)

```
    adj = calloc(V, sizeof(Aresta *));
    for (int k = 0; k < E; k++) {
        char a[2], b[2];
        if (scanf("%ls %ls", a, b) != 2) return 1;
        int u = letra_para_vertice(a[0], nomes), v = letra_para_vertice(b[0], nomes);
        liga(u, v);
        liga(v, u);                              // não direcionado: duas entradas
    }
    if (scanf("%d", &s) != 1) return 1;
    int *d = malloc(V * sizeof(int)), *fila = malloc(V * sizeof(int));
    for (int u = 0; u < V; u++) d[u] = -1;      // -1 = infinito (branco, não descoberto)
    int ini = 0, fim = 0;
    d[s] = 0;
    fila[fim++] = s;
    while (ini < fim) {                          // BFS: fila FIFO
```

PRÁTICA EM C

Busca em largura em C (4/4)

```
43          int u = fila[ini++];
44          for (Aresta *a = adj[u]; a; a = a->prox)
45              if (d[a->v] < 0) {                // descobre v: distância de u mais 1
46                  d[a->v] = d[u] + 1;
47                  fila[fim++] = a->v;
48              }
49      }
50      for (int u = 0; u < V; u++) {
51          if (d[u] < 0) printf("%c:inf ", nomes[u]);
52          else printf("%c:%d ", nomes[u], d[u]);
53      }
54      printf("\n");
55      return 0;
56  }
```



Experimento

- Operações contadas: vértices que saem da fila + entradas de adjacência
- Grafos com até 2 milhões de vértices
- Distância média com grau médio 10

EXPERIMENTO

Linear em $V + E$, e o mundo é pequeno

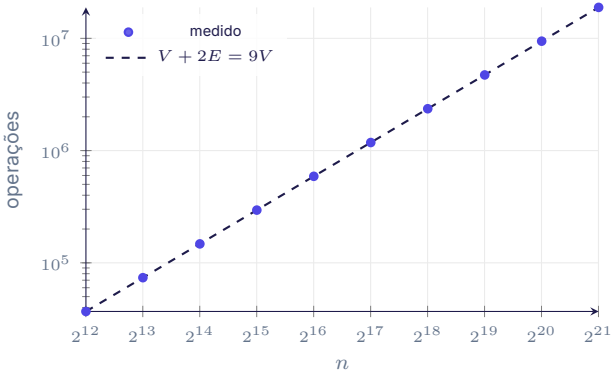
Hipótese. Num grafo não direcionado conexo, a BFS retira cada vértice da fila uma vez e examina cada entrada de adjacência uma vez: exatamente $V + 2E$ operações. Com $E = 4V$: $9V$. Num grafo aleatório com grau médio d, as camadas crescem como d^k: a distância típica é $\approx \ln V / \ln d$, que cresce **muito devagar**.

Método

1. Listas compactas (CSR) para economizar memória
2. $V = 2^{12}$ a 2^{21}, $E = 4V$, com um caminho que garante conexidade
3. Distância: grafo aleatório com $E = 5V$ (grau médio 10)
4. Contagem exata das operações

EXPERIMENTO

Operações da BFS



n	medido	razão
4.096	36.864	1,000
16.384	147.456	1,000
65.536	589.824	1,000
262.144	2.359.296	1,000
1.048.576	9.437.184	1,000
2.097.152	18.874.368	1,000

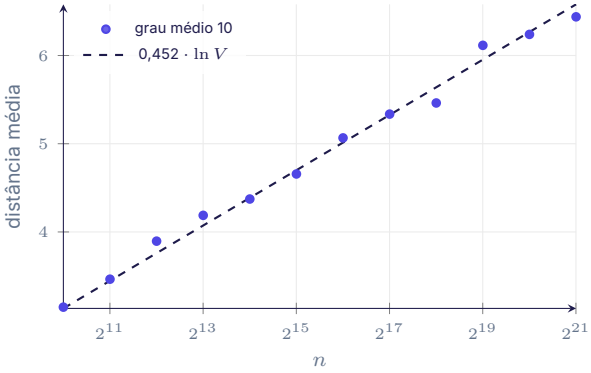
razão = medido $\div\ V + 2E = 9V$

✔ desvio máx. 0,0%

Exatamente $9V$ em todos os tamanhos: o custo é linear no tamanho do grafo.

EXPERIMENTO

Distância média: seis graus de separação



n	medido	razão
1.024	3,148	0,454
4.096	3,896	0,468
16.384	4,373	0,451
65.536	5,066	0,457
262.144	5,462	0,438
2.097.152	6,438	0,442

razão = medido ÷ $\ln V$

✔ desvio máx. 3,2%

Dois milhões de vértices a uma distância média de 6,4. A constante ajustada fica perto de $1/\ln 10 = 0{,}43$.

EXPERIMENTO

O que o experimento mostrou

A BFS faz exatamente $V + 2E$ operações: **linear**. E grafos aleatórios são “pequenos”: com dois milhões de vértices e grau médio 10, a distância média é 6,4, crescendo como $\ln V$. É o fenômeno dos **seis graus de separação**, medido por BFS em redes sociais reais.

Reproduza: `gcc -O2 aula28_experimento.c -o exp -lm && ./exp`



Conexões

- Redes sociais
- Web e buscadores
- Redes de computadores
- Jogos e robótica

CONEXÕES

Onde isso aparece na computação

Redes sociais

O Facebook mediu, com BFS em 721 milhões de usuários, uma distância média de 4,74; “pessoas que você talvez conheça são vértices à distância 2.

Web e buscadores

Rastreadores (*crawlers*) percorrem a web em largura a partir de páginas-semente: a BFS encontra primeiro as páginas mais próximas e relevantes.

Redes de computadores

Broadcast e descoberta de vizinhos inundam a rede camada por camada; o `traceroute` revela as camadas da BFS da internet.

Jogos e robótica

Em mapas em grade sem pesos, a BFS dá o menor caminho de um personagem ou robô; o cubo de Rubik foi resolvido (“número de Deus = 20) com buscas em largura.

PARA CASA

Para casa

Exercícios

1. (22.1-1) Com listas de adjacência, quanto custa calcular o grau de saída e o de entrada de todos os vértices?
2. (22.1-5) Calcule o quadrado de um grafo ($u \to w$ se há caminho de 2 arestas) com listas e com matriz.
3. (22.2-1) Execute a BFS na Figura 22.2(a) a partir do vértice 3.
4. (22.2-7) Divida lutadores em “bons e “maus com rivalidades entre grupos: é testar se o grafo é bipartido. Faça em $O(V + E)$.
5. Baixe uma rede real (SNAP, Stanford) e meça a

Leitura
Cormen et al., seções 22.1 e 22.2.

Código
josedihego.net/estruturas